

Gymnázium J. K. Tyla



Agentický Inženýrství - DarkFactory: pipeline pro automatizaci softwarového vývoje

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D

Vedoucí práce: Michal Dočekal

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Práce zkoumá přechod od konverzační asistence k delegovanému agentnímu vývoji a roli harnessu jako běhového a integračního prostředí coding agenta. Agentické inženýrství vymezuje jako soubor postupů, které činí AI-asistovaný vývoj softwaru účinným, kontrolovaným, opakovatelným a škálovatelným. DarkFactory představuje praktický artefakt tohoto přístupu. Dostupná implementační evidence zatím podporuje pouze vybrané mechanismy; konečné vyhodnocení je podmíněno ověřením jedné pinované revize a reprodukovatelných důkazů.

Klíčová slova: jazykové modely; coding agents; harness; Agentické inženýrství; GitHub Actions; DarkFactory

Abstract

This thesis examines the transition from conversational assistance to delegated agentic software development and the role of the harness as the coding agent's execution and integration environment. It defines Agentic Engineering as a set of practices that make AI-assisted software development effective, controlled, repeatable, and scalable. DarkFactory is presented as a practical artefact of this approach. The currently available implementation evidence supports only selected mechanisms; final evaluation depends on verification against one pinned revision and reproducible evidence.

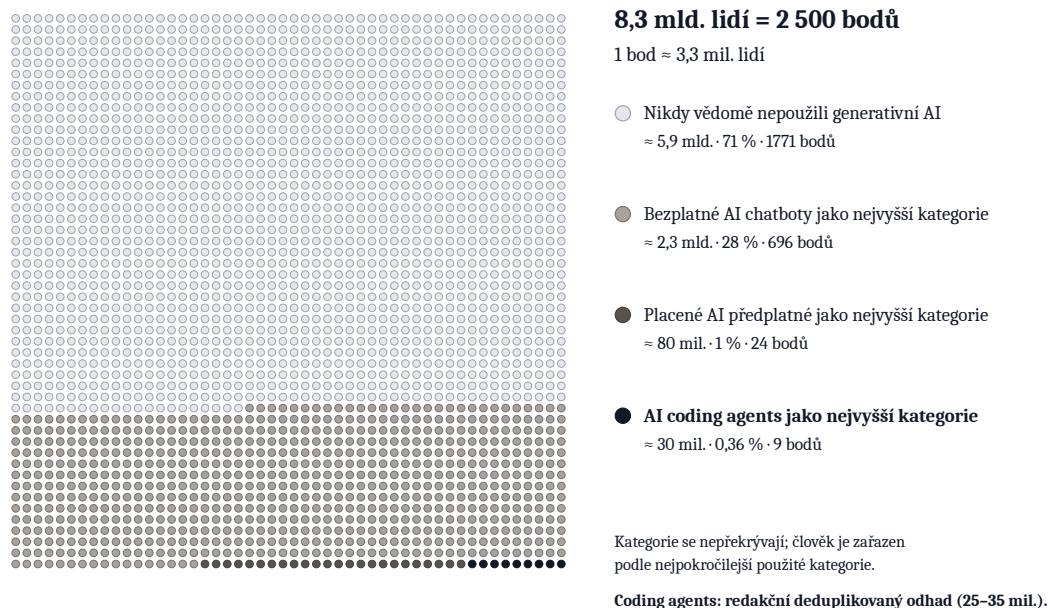
Keywords: language models; coding agents; harness; Agentic Engineering; GitHub Actions; DarkFactory

Obsah

Prohlášení	2
Anotace	3
1 Úvod	5
1.1 Cíl a vymezení	5
2 Teoretická část	6
2.1 Jazykový model v agentním systému	6
2.1.1 Context window a kompakce	6
2.2 Agent a harness	7
2.2.1 Smyčka	7
2.3 Agentické inženýrství	8
2.3.1 Zadání a kontext	8
2.3.2 Orchestrace a lidská integrace	8
3 Praktická část	10
3.1 Metodika	10
3.2 DarkFactory	11
3.2.1 Spuštění a kontejnerizované prostředí	11
3.2.2 Fázovaný průchod a řídicí smyčka	12
3.2.3 Serializace stavu, checkpointing a obnova	12
4 Výsledky a diskuse	13
4.1 Zjištění	13
4.2 Diskuse	14
5 Závěr	16
Seznam zdrojů	17
Seznam obrázků a tabulek	20

1 Úvod

Nástroje založené na velkých jazykových modelech prošly rychlým vývojem: od doplňování kódu při psaní přes konverzační chatboty až po autonomní agenty, kteří pomocí nástrojů samostatně provádějí změny v repozitáři a spouštějí příkazy ⁽¹⁻⁵⁾. S rostoucími schopnostmi modelů roste i jejich adopce, avšak pravidelné využívání plnohodnotných agentických systémů zůstává omezeno na přibližně 0,36 % světové populace ⁽⁶⁾.



Obrázek 1: Odhad rozdělení uživatelů generativní AI podle nejpokročilejší používané kategorie. Pravidelní uživatelé AI coding agents tvoří podle Gradually přibližně 0,36 % světové populace, tedy zhruba 30 milionů lidí ⁽⁶⁾.

Aby mohl agent samostatně pracovat na projektu, nestačí pouhé generování odpovědí. Potřebuje kontext z repozitáře, přístup k prostředí, nástroje pro spouštění testů a CI, uchování stavu mezi jednotlivými kroky a vymezený bod, v němž člověk rozhodne o přijetí výsledku ^(7,8).

1.1 Cíl a vymezení

Cílem práce je ukázat, jak harness dělá z jazykového modelu autonomního agenta a jaké postupy umožňují využívat agenty účinně a kontrolovaně.

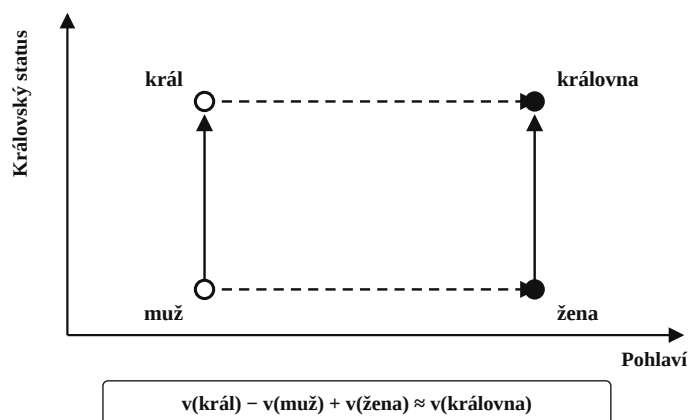
Praktická část analyzuje DarkFactory, na poskytovateli nezávislou pipeline pro AI-asistovaný softwarový vývoj implementovanou v jazyce Python.

2 Teoretická část

2.1 Jazykový model v agentním systému

Jazykový model (LLM) předpovídá další token na základě **kontextu**. **Transformer** zpracovává vztahy mezi nimi pomocí mechanismu attention ^(9,10). Při inferenci model zpracuje obsah kontextového okna a vytvoří posloupnost výstupních tokenů. Jednotlivé volání však samo nevybírám další kontext, nemění soubory, nespouští příkazy ani neuchovává stav mezi kroky. Tyto činnosti zajišťuje harness, který modelu zpřístupňuje nástroje a jejich výsledky vrací do dalšího kroku ⁽¹¹⁾.

Vektorové reprezentace, označované jako **embeddingy**, zachycují sémantické vztahy ve vektorovém prostoru. Známým příkladem je vztah mezi vektory slov king a queen ⁽¹²⁾.



Obrázek 2: Jednoduchá ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov **king** a **queen** podle principu popsaného v ⁽¹²⁾.

2.1.1 Context window a kompakce

Kontextové okno (context window) tvoří pracovní kontext jednoho volání modelu. Může obsahovat instrukce, části repozitáře, historii volání nástrojů i výsledky předchozích kroků. Jeho kapacita však sama o sobě nezaručuje, že model všechny podstatné informace správně využije: úspěšnost jejich vybavení závisí také na umístění v kontextu a může s rostoucí délkou vstupu klesat ⁽¹³⁾. Toto postupné zhoršování práce s nahromaděným kontextem se označuje jako **context rot** ⁽¹⁴⁾.

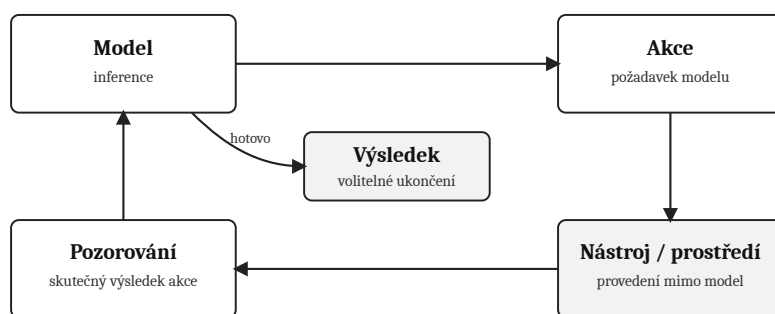
Kompakce po překročení stanoveného limitu nahrazuje starší průběh strukturovaným souhrnem klíčových rozhodnutí a dosažených výsledků. Do dalšího volání tak není nutné vkládat celý přepis předchozí interakce ⁽¹⁴⁾.

2.2 Agent a harness

Zásadní rozdíl mezi konverzačním chatbotem a autonomním agentem nespočívá v architektuře použitého modelu, nýbrž v míře delegace provádění. Chatbot setrvává v roli externího rádce: uživatel musí manuálně kopírovat úryvky kódu, dodávat kontext a spouštět navržené příkazy. Agent naproti tomu prostřednictvím harnessu získává přímý přístup k nástrojům repozitáře. Sám prochází souborovou strukturu, upravuje kód, spouští testy a na základě chybových výstupů samostatně koriguje své změny ^(11, 15). Tento posun transformuje roli člověka z přímého vykonavatele na dohlížejícího architekta.

2.2.1 Smyčka

Základním mechanismem agentického systému je iterativní řídicí smyčka. Na rozdíl od jednorázové generace odpovědi u chatbota probíhá interakce v cyklu podle vzoru **ReAct** (**Reasoning and Acting**) ⁽¹⁶⁾. Model v každém kroku analyzuje aktuální stav kontextu, zformuluje vnitřní uvažování a navrhne konkrétní volání nástroje ve formě strukturovaného požadavku. Harness tuto akci provede v prostředí projektu, zachytí výsledek a vrátí jej modelu jako nové pozorování. Smyčka pokračuje, dokud model nedosáhne cíle nebo nenarazí na bezpečnostní limit kroků či tokenů.



Obrázek 3: Agentní smyčka ReAct: model navrhne akci, harness ji zprostředkuje a vykoná v prostředí a pozorování se vrací do dalšího kroku; princip podle ⁽¹⁶⁾.

2.3 Agentické inženýrství

Agentické inženýrství (**Agentic Engineering**) představuje soubor postupů pro systematický návrh podmínek, v nichž může jazykový model vykonávat inženýrské úlohy účinně, kontrolovaně, opakovatelně a v souladu s pravidly projektu ^(7,8). Zahrnuje dekompozici úloh, přesné ohraničení kontextu, konstrukci nástrojových rozhraní, verifikační zpětnou vazbu a mechanismy řízené integrace.

2.3.1 Zadání a kontext

Spolehlivé delegování práce začíná explicitním vymezením cíle, rozsahu, omezení a podmínek přijetí. Specifikace popisuje nejen požadovaný výsledek, ale také části systému, které se měnit nemají, a způsob, jakým bude výsledek ověřen. Tento přístup, označovaný jako **spec-first** nebo **spec-driven development**, dává agentovi před implementací měřitelné hranice a člověku podklad pro posouzení výsledku.

Prompt engineering se soustředí na formulaci instrukcí, omezení, příkladů a očekávaného výstupu konkrétního inferenčního kroku. **Context engineering** řeší širší a průběžný výběr, uspořádání, obnovování a kompakci informací, které má model v daném kroku k dispozici ^(14,17). Projektová pravidla a validační příkazy lze uchovat přímo v repozitáři například ve standardizovaném souboru `AGENTS.md`, takže je harness může připojit k relevantní úloze ⁽¹⁸⁾.

Specifikace a kontext se promítají do provádění prostřednictvím schopností harnessu. **Nástroje** umožňují agentovi číst a upravovat soubory, vyhledávat nebo spouštět příkazy; **Skills** spojují opakovaně použitelné instrukce, skripty a zdroje pro určitý typ úlohy ⁽¹⁹⁾. **Hooks** reagují na události životního cyklu a mohou před akcí či po ní vynutit deterministickou kontrolu ⁽²⁰⁾. **Model Context Protocol (MCP)** standardizuje napojení externích nástrojů a datových zdrojů prostřednictvím rozhraní klient–server ⁽²¹⁾. Nejde o samostatné agenty, ale o řízené části runtime, jejichž výběr, oprávnění a výstupy určují, co může agent skutečně provést a ověřit.

Po každé akci harness vrací agentovi pozorování, například výsledek testu, překladače nebo stav pracovního stromu. Agent podle něj upraví další krok a pokračuje, dokud nesplní podmínky přijetí nebo nenarazí na stanovený limit. Takový **goal loop** mění jednorázové generování v řízený proces, v němž verifikační zpětná vazba průběžně koriguje další postup ^(7,16).

2.3.2 Orchestrace a lidská integrace

Rozsáhlou úlohu lze rozdělit do více agentních běhů, pokud mají podúlohy jasné hranice a jejich výsledky lze znovu integrovat. Ve vzoru **coordinator/subagent** koordinátor deleguje dílčí úkol specializovanému subagentovi s vlastním kontextem a přebírá jeho výsledek.

Nezávislé podúlohy mohou zpracovat **paralelní pracovníci**, zatímco **workflow graph** předem určuje závislosti, pořadí a větvení fází. **Swarm** používá volnější koordinaci, při níž si specializovaní agenti dynamicky předávají řízení podle aktuálního stavu úlohy ^(22, 23).

Více agentů samo o sobě nezaručuje lepší výsledek. Paralelizace přináší užitek jen tehdy, když jsou omezeny vzájemné závislosti a koordinátor dokáže odhalit konflikty, ověřit dílčí výstupy a posoudit sloučený výsledek vůči společným podmínkám přijetí. Orchestrace proto zahrnuje nejen rozdělení práce, ale také správu kontextu, pořadí kroků, sdíleného stavu a integračních kontrol.

Princip **human-in-the-loop (HITL)** doplňuje automatizované smyčky o kontrolní brány, v nichž je vyžadováno explicitní lidské rozhodnutí, například schválení specifikace, potvrzení implementačního plánu nebo přijetí výsledného diffu. Vývojář tak nadále odpovídá za záměr, architekturu a integraci, zatímco agent provádí ohraničenou implementační práci. Začlenění změn do hlavní větve vyžaduje vedle technického ověření v CI také lidskou revizi a převzetí odpovědnosti za výsledek ^(24, 25).

3 Praktická část

3.1 Metodika

Praktická část práce navazuje na teoretická východiska o harnessu a Agentickém inženýrství. Jejím cílem je empiricky ověřit, jakým způsobem jsou principy řízeného provádění, ohraničení kontextu a verifikace realizovány v praxi. Předmětem zkoumání je referenční revize systému DarkFactory ⁽²⁶⁾, která představuje stabilní implementaci autonomní vývojové pipeline v ekosystému GitHub Actions.

Analýza se zaměřuje na čtyři klíčové inženýrské dimenze odvozené z teoretické části:

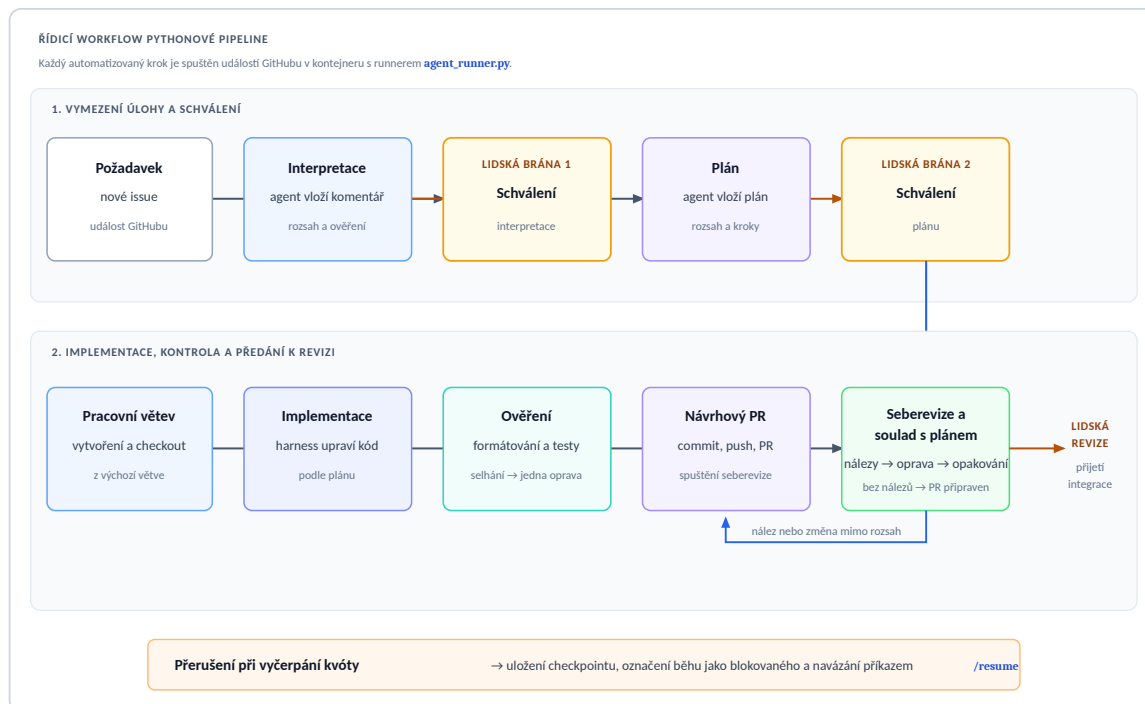
1. **Izolace běhového prostředí a oprávnění:** mechanismus hermetického oddělení agenta od hostitelského CI systému, montování pracovního stromu repozitáře a injektování autentizačních tajemství s minimálními právy.
2. **Dekompozice úlohy a fázované řízení (HITL):** rozdělení životního cyklu požadavku do diskrétních stavů (interpretace, plánování, implementace) a začlenění formálních schvalovacích bran člověkem.
3. **Deterministická verifikace a seberevize:** integrace automatizovaných nástrojů (lintery, testovací frameworky), záchyt chybových kódů v samoopravné smyčce a auditování vygenerovaného diffu vůči schválenému plánu.
4. **Perzistence stavu a zotavení:** serializace parametrů běhu (checkpointing), záchyt výpadků kvót externích API (HTTP 429) a procedura obnovení rozpracované úlohy příkazem /resume.

Metodický postup kombinuje statickou inspekci zdrojových kódů a reprodukovatelnou verifikaci testovací sady. Primárními podklady jsou specifikace workflow (`.github/workflows/agent.yml`), definice kontejneru (`docker/Dockerfile.agent`), řídicí skript runneru (`agent_runner.py`) a sada integračních testů v jazyce Python (`tests/test_governance.py`, `tests/test_pipeline_config.py`, `tests/test_workflows.py`). Tvzení o strukturálním uspořádání a řídicích tocích vycházejí z trasování kódu runneru; spolehlivost přechodů mezi stavy, obnova po přerušení a kontrola rozsahu oprávnění jsou ověřeny vykonáním automatizovaných testů v referenčním prostředí.

Výzkumný rámec je vymezen zkoumáním samotné pythonovské pipeline a jejích řídicích mechanismů. Analýza nehodnotí subjektivní kvalitu kódu generovaného konkrétními modely, nýbrž strukturální a invariantní vlastnosti běhového prostředí, které vymezují bezpečný prostor pro autonomní agentní práci.

3.2 DarkFactory

DarkFactory představuje automatizovanou pipeline běžící v prostředí GitHub Actions. Workflow spouští kontejner Docker, v němž pythonovský runner připraví prostředí, předá harnessu instrukce a zpracuje výsledek jeho běhu ⁽²⁶⁾.



Obrázek 4: Řídící workflow Pythonové pipeline DarkFactory. Schéma zachycuje průchod od nového požadavku přes dvě schvalovací brány, implementaci a opakovanou seberevizi až po lidskou revizi pull requestu; větve přerušení odpovídá zachování checkpointu a obnovení běhu ⁽²⁶⁾.

3.2.1 Spuštění a kontejnerizované prostředí

Celý životní cyklus úlohy je vázán na webhooks repozitáře. Pipeline je spouštěna událostmi otevření issue nebo přidání komentáře (`issues.opened`, `issue_comment.created`). Úvodní krok workflow v GitHub Actions nejprve ověřuje oprávnění autora události — odmítá vstupy generované automatickými boty a vyžaduje oprávnění k zápisu do repozitáře.

Po validaci vstupu workflow sestaví izolovaný obraz Docker na základě předpisu `docker/Dockerfile.agent`. Tento obraz obsahuje základní systémové nástroje, interpret jazyka Python, verzovací systém git a potřebné vývojové knihovny. Pracovní kopie repozitáře je do kontejneru připojena jako dedikovaný svazek (volume mount). Tím je zaručeno, že agent pracuje výhradně nad ohraničeným pracovním stromem a nemá přímý přístup k hostitelskému systému CI runneru. Přístupové tokeny GitHub API i klíče k externím modelům jsou injektovány do kontejneru jako přísně ohraničené proměnné prostředí s minimálními nezbytnými oprávněními ⁽²⁶⁾.

3.2.2 Fázovaný průchod a řídicí smyčka

Řízení úlohy v kontejneru přebírá skript `agent_runner.py`. Průchod není realizován jako jediná dlouhá interakce, nýbrž je rozdělen do fází oddělených kontrolními branami:

- **Fáze interpretace:** Runner načte text issue a předloží jej modelu se systémovou instrukcí pro dekompozici problému. Model zformuluje pochopení úlohy a navrhne akceptační kritéria. Runner výsledek zapíše jako komentář k issue a ukončí běh, čímž vyčká na schválení člověkem.
- **Fáze plánování:** Po explicitním schválení interpretace je workflow spuštěno znovu. Model analyzuje strukturu repozitáře a sestaví konkrétní plán úprav — specifikuje seznam souborů určených ke změně a předpokládaný sled kroků. Tento plán je opět předložen člověku ke schválení.
- **Implementační smyčka a verifikace:** Teprve po schválení plánu runner vytvoří dedikovanou pracovní větev, aby nezasahoval do hlavní vývojové větve. Harness následně model navádí k úpravě kódu. Po dokončení úprav runner automaticky spustí deterministické ověřovací nástroje: linter a sadu jednotkových testů. V případě detekce syntaktické chyby či pádu testu má agent k dispozici přesný chybový výstup a jeden vyhrazený krok na samoopravu.
- **Seberevize a draft pull request:** Jakmile testy projdou, runner vytvoří commit, odešle větev na vzdálený server a otevře draft pull request. Následně proběhne fáze seberevize: runner vygeneruje diff mezi pracovní větví a výchozím stavem a porovná změněné soubory se schváleným rozsahem plánu. Pokud model či kontrolní skript odhalí změny mimo povolený rozsah, vyžádá se náprava. Při shodě je pull request označen jako připravený k lidské revizi ⁽²⁶⁾.

3.2.3 Serializace stavu, checkpointing a obnova

V distribuovaném prostředí automatizovaných pipeline představují limity zdrojů (GitHub Actions timeout, kvóty externích poskytovatelů LLM či síťové výpadky) zásadní výzvu pro spolehlivost. DarkFactory tento problém řeší explicitní serializací stavu běhu (checkpointing).

Během každé fáze runner ukládá run state do strukturovaného formátu. Stavový záznam obsahuje identifikátor požadavku, aktuální fázi, název vytvořené větve, hash posledního commitu a přehled úspěšně dokončených kroků. Narazí-li volání API modelu na limit vyčerpání kvóty (HTTP 429), runner neselže fatální chybou; namísto toho bezpečně uloží dosažený stav, označí úlohu v komentáři jako blokovanou a informuje vývojáře. Jakmile dojde k obnovení kvóty nebo přepnutí klíče, lze pipeline probudit příkazem `/resume`. Runner deserializuje uložený checkpoint a naváže přesně v místě přerušení, aniž by opakoval již provedené kroky nebo zbytečně znovu spotřebovával tokeny pro již vyřešené úlohy

^(26, 27)

4 Výsledky a diskuse

4.1 Zjištění

Pipeline organizuje průchod požadavku jako sled událostí v repozitáři. Po vytvoření issue runner vygeneruje interpretaci zadání; její schválení uživatelem iniciuje tvorbu plánu a teprve další schválení zahájí samotnou implementaci. Dvě schvalovací brány tak oddělují porozumění požadavku a plánování od samotných úprav zdrojového kódu ⁽²⁶⁾.

Workflow před spuštěním ověřuje, zda je agent pro daný repozitář povolen, a ignoruje komentáře vytvořené automatickými boty. Autentizační tokeny jsou předány z prostředí GitHub Actions a pracovní strom projektu je připojen do kontejneru, v němž běží pythonovský runner ⁽²⁶⁾.

Po schválení plánu pipeline vytvoří pracovní větev a předá implementační instrukci harnessu. Runner spustí formátovací a testovací nástroje; při neúspěchu testů provede opravný krok. Poté vytvoří commit, odešle větev a otevře draft pull request. Seberevize porovná změněné soubory s rozsahem plánu; pokud detekuje nežádoucí úpravy mimo schválený rozsah, iniciuje opravu a novou revizi. Jestliže seberevize nenajde nesoulad, pipeline označí pull request jako připravený k lidské revizi ⁽²⁶⁾.

Inspekce testovací sady v referenční revizi dokládá vysoké pokrytí klíčových invariantů pipeline: automatizované testy ověřují korektní serializaci Run State, detekci zastaralého plánu (stale plan), deterministické odvození stavu pracovního stromu i dodržení bezpečnostních mantinelů při práci se soubory. Testované scénáře potvrzují, že při selhání jednotkových testů je chybový výstup linteru či testovacího frameworku přesně strukturován a předán modelu, což umožňuje úspěšné odstranění lokálních regresí bez lidského zásahu ⁽²⁶⁾.

Runner ukládá checkpoint pro spolehlivé navázání rozpracovaného běhu. Při vyčerpání kvóty uchová stav, označí běh jako blokový a umožní jeho pokračování příkazem / resume. Testy pokrývají serializaci stavu, obnovení pipeline, přepnutí poskytovatele modelu i kontrolu přístupových oprávnění ⁽²⁶⁾.

4.2 Diskuse

Dostupný snapshot dokládá některé mechanismy relevantní pro pozdější vyhodnocení, zejména práci s trvalým stavem, oprávněními, izolací změn, automatickými kontrolami a vybranými scénáři obnovy. Neměří však účinnost jednotlivých postupů Agentického inženýrství, kvalitu práce s kontextem ani úplnost orchestrace. Konečné vyhodnocení cíle proto vyžaduje potvrzení kanonické revize DarkFactory a reprodukovatelný evidenční manifest; následující interpretace jsou z tohoto důvodu předběžné.

Dělbá odpovědnosti mezi modelem a harnessem. Jazykový model sám o sobě postrádá pojem o čase, kauzalitě i stavu vývojového prostředí; funguje jako stochastický generátor návrhů. Výsledky ukazují, že skutečnou páteř autonomního systému tvoří deterministický harness — v tomto případě GitHub Actions workflow a pythonovský runner. Právě harness zodpovídá za přípravu izolovaného prostředí, vynucování kroků, orámování kontextu a interpretaci návratových kódů. Model tedy nepředstavuje samostatnou autonomní entitu, nýbrž výpočetní modul zasazený do přísně strukturovaného algoritmického rámce ⁽⁷⁾.

Architektonický kompromis: Autonomie vs. správa. DarkFactory volí konzervativní architekturu se čtyřmi kontrolními branami — dvěma lidskými (schválení interpretace a plánu) a dvěma automatizovanými (testy a audit diffu). Tento přístup eliminuje nejčastější rizika nekontrolovaných agentů: drift zadání, neautorizované zásahy do konfigurací a nekonečné cykly generování kódu. Cenou za vysokou kontrolu je však fragmentace autonomie a vznik lidského úzkého hrdla. Proces nepředstavuje bezobslužný vývojový systém, nýbrž asistovanou výrobní linku, kde člověk plní roli architekta a revizora ⁽⁸⁾.

Ekonomika kontextu a perzistence stavu. V souladu s poznatkami o degradaci pozornosti při dlouhém kontextu ⁽¹³⁾ DarkFactory neudržuje historii celého projektu uvnitř context window. Verzovací systém git a externí checkpointy slouží jako primární nositelé stavu. Do context window vstupují při každém kroku pouze minimální nezbytné informace: zadání, schválený plán a bezprostřední pozorování z testů. Oddělení trvalého stavu od dočasného kontextu inference snižuje náklady na tokeny a udržuje pozornost modelu zaměřenou na řešený fragment kódu.

Deterministická verifikace jako protiváha stochastické generace. Softwarové inženýrství nabízí pro nasazení agentů zásadní výhodu: možnost objektivní, strojově ověřitelné zpětné vazby překladačů, linterů a jednotkových testů. Praktická analýza potvrdila, že samoopravná smyčka agenta funguje spolehlivě pouze tehdy, má-li model k dispozici přesný chybový výstup (stack trace, exit code) a pevně vymezený rozsah povolených úprav.

Bez těchto deterministických mantinelů hrozí, že pokus o opravu chyby vnese do repozitáře další nezamýšlené regrese.

5 Závěr

Jazykový model sám o sobě pouze generuje výstup v závislosti na vstupním kontextu. Agent z něj vzniká až propojením s nástroji, prostředím, stavem a pozorováním, které zajišťuje harness ^(7, 11). Ani toto propojení však pro účinné nasazení ve vývoji softwaru nestačí bez postupů Agentického inženýrství, které vymezují rozsah autonomie, určují body lidského rozhodnutí a zajišťují ověřování výsledků.

Analýza pipeline DarkFactory doložila, jak lze takový systém realizovat v praxi: řídí průchod požadavku, vytváří izolovanou větev, ověřuje výsledek testy, kontroluje rozsah změn, ukládá checkpoint a předává výsledek k lidské revizi.

Úloha vývojáře se v tomto uspořádání posouvá od rutinního psaní kódu k preciznímu návrhu zadání, vymezení mantinelů a určení bodů, v nichž je nezbytné lidské rozhodnutí.

Zjištění ukazují, že samotná autonomie agenta nenahrazuje řízení vývojového procesu. Správně navržený harness spolu s postupy Agentického inženýrství umožňuje delegovat netriviální vývojové úlohy na stochastické modely a současně chránit stabilitu hlavní vývojové větve i transparentnost provedených změn. Rozsah této práce však neumožňuje zobecnit výsledky na jiné harnessy ani posoudit kvalitu kódu vytvářeného různými modely.

Seznam zdrojů

1. GITHUB. GitHub Copilot code suggestions in your IDE. Online. 2026. [Accessed 22 září 2026]. Available from: <https://docs.github.com/en/copilot/concepts/completions/code-suggestions>
2. GITHUB. Asking GitHub Copilot questions in your IDE. Online. 2026. [Accessed 22 září 2026]. Available from: <https://docs.github.com/en/copilot/how-tos/chat-with-copilot/chat-in-ide>
3. GITHUB. Use GitHub Copilot agents. Online. 2026. [Accessed 22 září 2026]. Available from: <https://docs.github.com/en/copilot/how-tos/use-copilot-agents>
4. OPENAI. Introducing Codex. Online. 2025. [Accessed 22 září 2026]. Available from: <https://openai.com/index/introducing-codex/>
5. OPENAI. Introducing the Codex app. Online. 2026. [Accessed 22 září 2026]. Available from: <https://openai.com/index/introducing-the-codex-app/>
6. HILLEBRANDT, Finn. How Many People Use AI? Facts & Figures 2026. Online. 2026. [Accessed 22 září 2026]. Available from: <https://www.gradually.ai/en/how-many-people-use-ai/Published-2026-08-23>
7. ANTHROPIC. Harness design for long-running application development. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/harness-design-long-running-apps>
8. ANTHROPIC. Scaling Managed Agents: Decoupling the brain from the hands. Online. 2026. [Accessed 21 září 2026]. Available from: <https://www.anthropic.com/engineering/managed-agents>
9. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
10. BROWN, Tom B., MANN, Benjamin, RYDER, Nick, SUBBIAH, Melanie, KAPLAN, Jared, DHARIWAL, Prafulla, NEELAKANTAN, Arvind, SHYAM, Pranav, SASTRY, Girish, ASKELL, Amanda a OTHERS. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33. Available from: <https://arxiv.org/abs/2005.14165>
11. ANTHROPIC. Building effective agents. Online. 2024. Available from: <https://www.anthropic.com/engineering/building-effective-agents>

12. MIKOLOV, Tomas, YIH, Wen-tau a ZWEIG, Geoffrey. Linguistic Regularities in Continuous Space Word Representations. In : *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Online. 2013. p. 746–751. Available from: <https://aclanthology.org/N13-1090/>
13. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
14. ANTHROPIC. Effective context engineering for AI agents. Online. 2025. [Accessed 22 září 2026]. Available from: <https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents>
15. OPENAI. Sandbox agents: Concepts | OpenAI Agents SDK. Online. 2026. [Accessed 22 září 2026]. Available from: <https://openai.github.io/openai-agents-js/guides/sandbox-agents/concepts/>
16. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
17. OPENAI. Prompt engineering. Online. 2026. [Accessed 22 září 2026]. Available from: <https://platform.openai.com/docs/guides/prompt-engineering>
18. AGENTIC AI FOUNDATION. AGENTS.md. Online. 2026. [Accessed 23 září 2026]. Available from: <https://agents.md/>
19. AGENT SKILLS. Agent Skills: an open format for extending AI agents. Online. 2025. [Accessed 22 září 2026]. Available from: <https://agentskills.io/home>
20. OPENAI. Lifecycle | OpenAI Agents SDK. Online. 2026. [Accessed 22 září 2026]. Available from: <https://openai.github.io/openai-agents-python/ref/lifecycle/>
21. MODEL CONTEXT PROTOCOL. Model Context Protocol 2026-07-28 specification: server features. Online. 2026. [Accessed 22 září 2026]. Available from: <https://modelcontextprotocol.io/specification/2026-07-28/server>
22. OPENAI. Agent orchestration | OpenAI Agents SDK. Online. 2026. [Accessed 21 září 2026]. Available from: https://openai.github.io/openai-agents-python/multi_agent/

23. OPENAI. Swarm: educational framework exploring lightweight multi-agent orchestration. Online. 2024. [Accessed 22 září 2026]. Available from: <https://github.com/openai/swarm>
24. GITHUB. Branches | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/reference/branches>
25. GITHUB. About pull requests | GitHub Docs. Online. 2026. [Accessed 21 září 2026]. Available from: <https://docs.github.com/en/pull-requests/get-started/about-pull-requests>
26. MARIUS, Patrik. DarkFactory commit e9c10221. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/commit/e9c10221b40589512d262a0edb95f709b923150c>
27. MARIUS, Patrik. Request #359: finish the df production engine and retire Python orchestration. Online. 2026. [Accessed 21 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory/issues/359>

Seznam obrázků a tabulek

Obrázek 1	Odhad rozdělení uživatelů generativní AI podle nejpokročilejší používané kategorie. Pravidelní uživatelé AI coding agents tvoří podle Gradually přibližně 0,36 % světové populace, tedy zhruba 30 milionů lidí ⁽⁶⁾	5
Obrázek 2	Jednoduchá ilustrace sémantického vztahu mezi vektorovými reprezentacemi slov king a queen podle principu popsaného v ⁽¹²⁾	6
Obrázek 3	Agentní smyčka ReAct: model navrhne akci, harness ji zprostředkuje a vykoná v prostředí a pozorování se vrací do dalšího kroku; princip podle ⁽¹⁶⁾	7
Obrázek 4	Řídící workflow Pythonové pipeline DarkFactory. Schéma zachycuje průchod od nového požadavku přes dvě schvalovací brány, implementaci a opakovanou seberevizi až po lidskou revizi pull requestu; větev přerušení odpovídá zachování checkpointu a obnovení běhu ⁽²⁶⁾	11